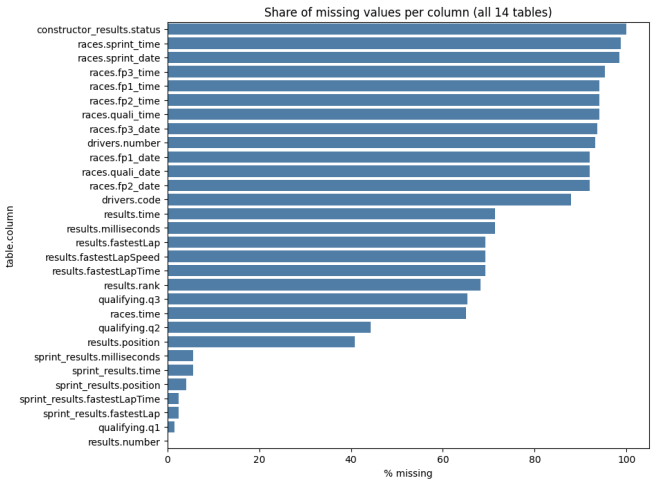
F1 Podium Prediction — MS1 Report

Notebook: kaggle.com/code/amreladawy2/f1-podium-prediction-ms1 · GitHub: github.com/amreladawy2/f1-podium-prediction-ms1 · Team (GitHub): amreladawy2, omaribrahim4761, Omarissa1309, Nadiemismail1

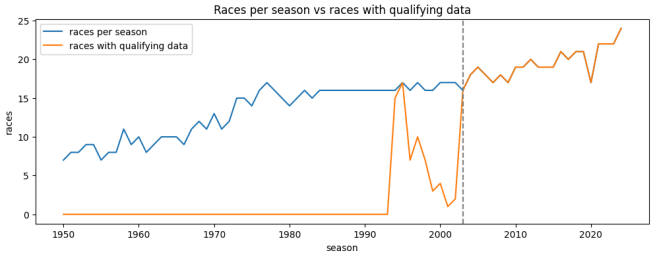
> **Task.** Predict whether a driver finishes on the podium (top 3) of a Formula 1 race, using only information that is known **after qualifying and before the race starts**. Data: Formula 1 World Championship 1950–2024 (Kaggle, rohanrao), 14 CSV tables, 26,759 result rows. **Result:** a tuned Random Forest reaches PR-AUC 0.684 and ROC-AUC 0.934 on the unseen 2022–24 seasons, compared with 0.474 / 0.793 for the simple rule "grid ≤ 3 → podium". **Bonus:** a better cleaning and split (V13, section 12) raises the test PR-AUC to 0.692 and F1 from 0.642 to 0.686.

1. EDA before cleaning

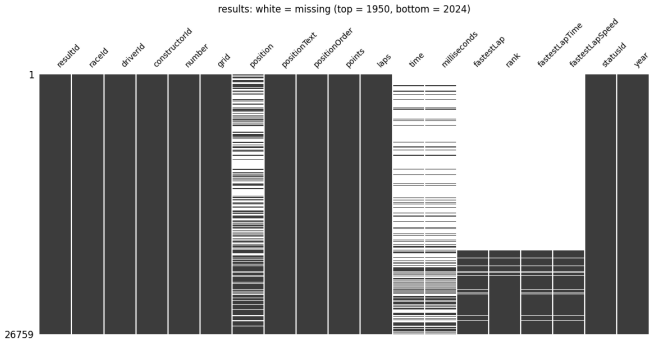
We first looked at the raw tables the Lab 1 way (`info`, `describe`, `isnull().sum()`, `value_counts`, `msno.matrix`) to find out what needs cleaning and which seasons can be used.



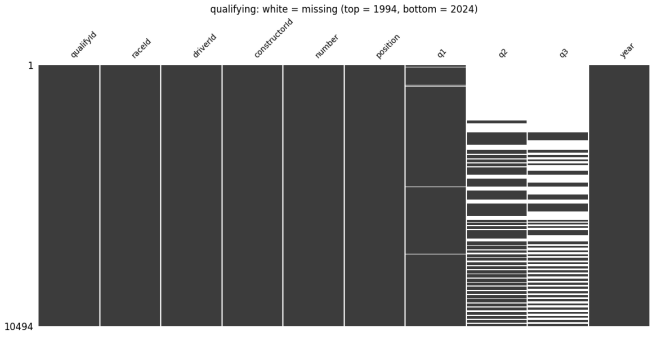
Missing share per column. The emptiest columns (constructor_results.status, practice/sprint dates, driver numbers and codes, fastest laps, race times) are post-race or not needed. q2/q3 are empty for drivers knocked out earlier and before 2006, so the qualifying gap uses the best of Q1–Q3.



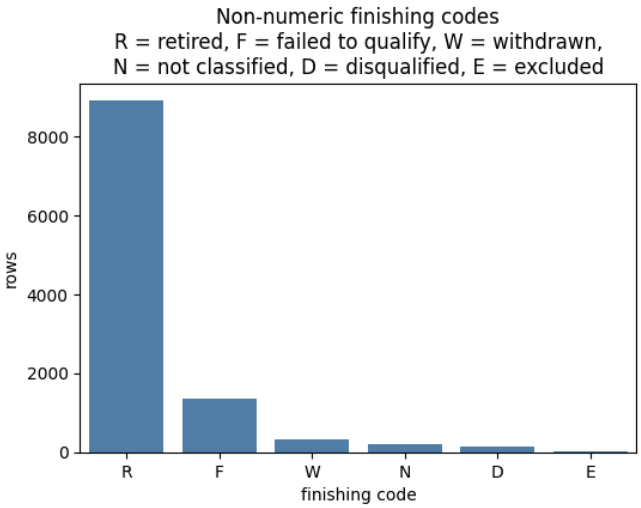
Races per season vs races with qualifying data. Qualifying is missing before 1994, patchy in 1996–2002 and complete from 2003, so the models use 2003 onwards.



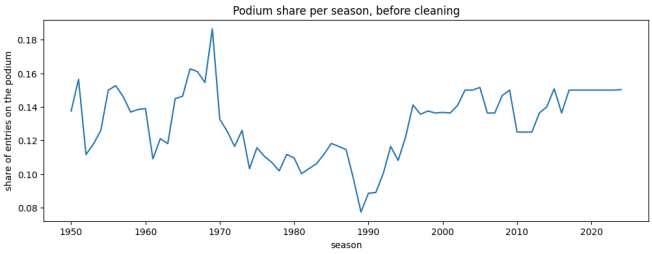
`msno.matrix` of results (rows ordered 1950 → 2024): fastest-lap columns only start in 2004.



`msno.matrix` of qualifying: q2/q3 only exist from 2006 (white block at the top) and are empty for drivers knocked out in an earlier session, by design.



Non-numeric finishing codes: F (failed to qualify) and W (withdrawn) never started, so they are removed.

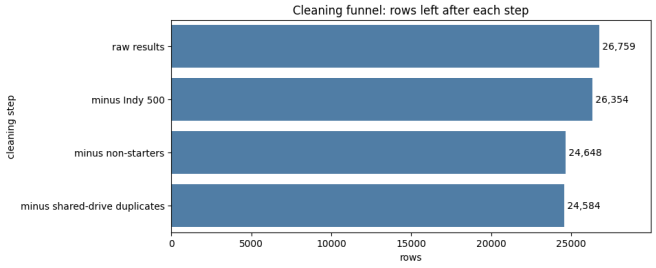


Raw podium share per season: low in the 1980s because up to ~39 entries competed for 26 grid slots.

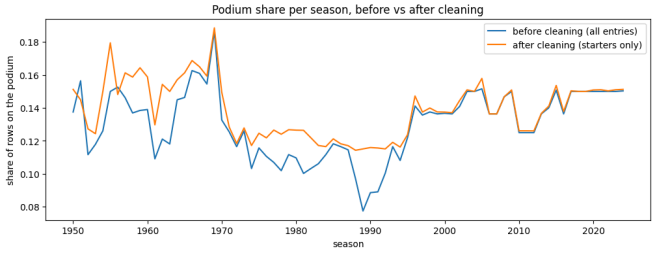
2. Data cleaning

The raw CSV files are never changed: `/kaggle/input` is read-only and every rule works on in-memory copies. Primary keys of all 14 tables (composite keys for lap_times and pit_stops) and 23 foreign keys were checked: 0 problems. Every cleaning step has a written reason and a before/after count in the notebook:

#	Rule	Effect
1	`\N` → missing; dates → datetime; lap-time strings → seconds	1 impossible qualifying time set to missing
2	Primary-key and foreign-key checks	all 14 primary keys unique, 0 orphans in 23 foreign keys
3	Remove the Indianapolis 500 (different series, shares a circuit with the US GP)	11 races, 405 rows
4	Remove entries that never started (F / W codes, not-started status with 0 laps)	1,706 rows; 24 "Withdrew" rows that actually raced kept
5	Shared drives (1950–1964): one row per (raceId, driverId), keep the car the driver started in	64 rows
6	grid = 0 → back of the grid + `pitlane_start` flag	87 rows
7	Target `podium = positionOrder ≤ 3`	podium share 13.6 % (14.4 % for 2003–24)
8	Championship standings shifted to the previous round (the stored standings already include the race)	removes a leak
9	Nationality → home country(ies); "United States" → "USA"	home-race flag
10	Status values grouped: finished / mechanical / ambiguous / accident / retired / other	used for Q3 and team reliability



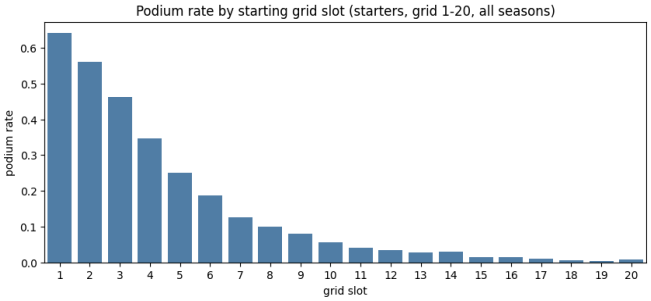
Cleaning funnel: 26,759 → 24,584 rows. Removing non-starters is the biggest step.



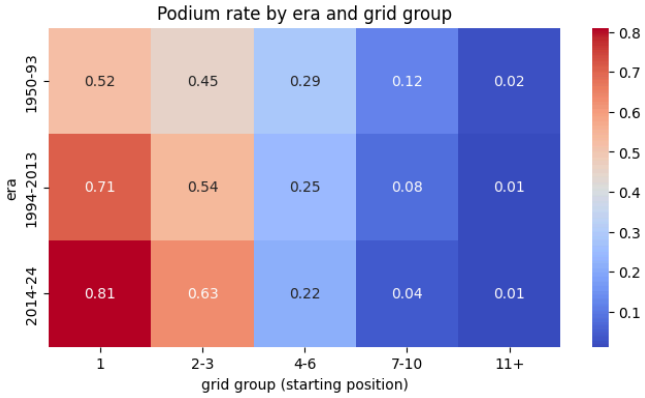
Podium share per season before vs after cleaning: the change is mainly in the 1980s–90s non-qualifier era.

3. EDA after cleaning

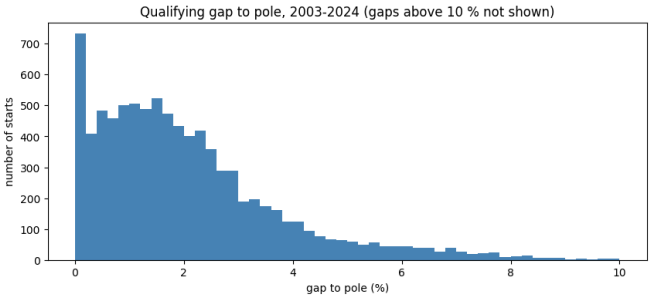
The cleaned table `df` has 24,584 rows, one per (raceId, driverId), and no missing values except the championship rank where no earlier standings exist (round 1 of a season, or a driver's first race).



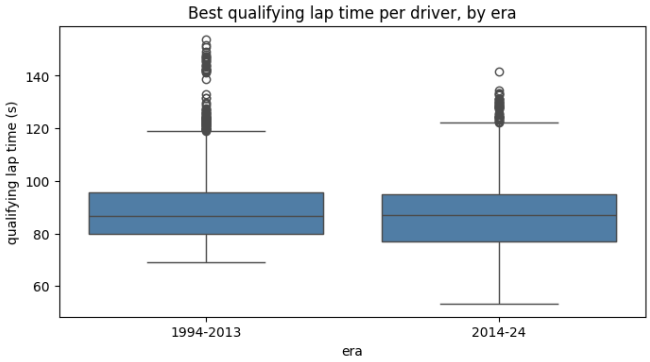
Podium rate by grid slot: P1 64 %, P2 56 %, P3 46 %, falling steeply after that.



The grid has become more decisive: pole converts 52 % (1950–93), 71 % (1994–2013), 81 % (2014–24).



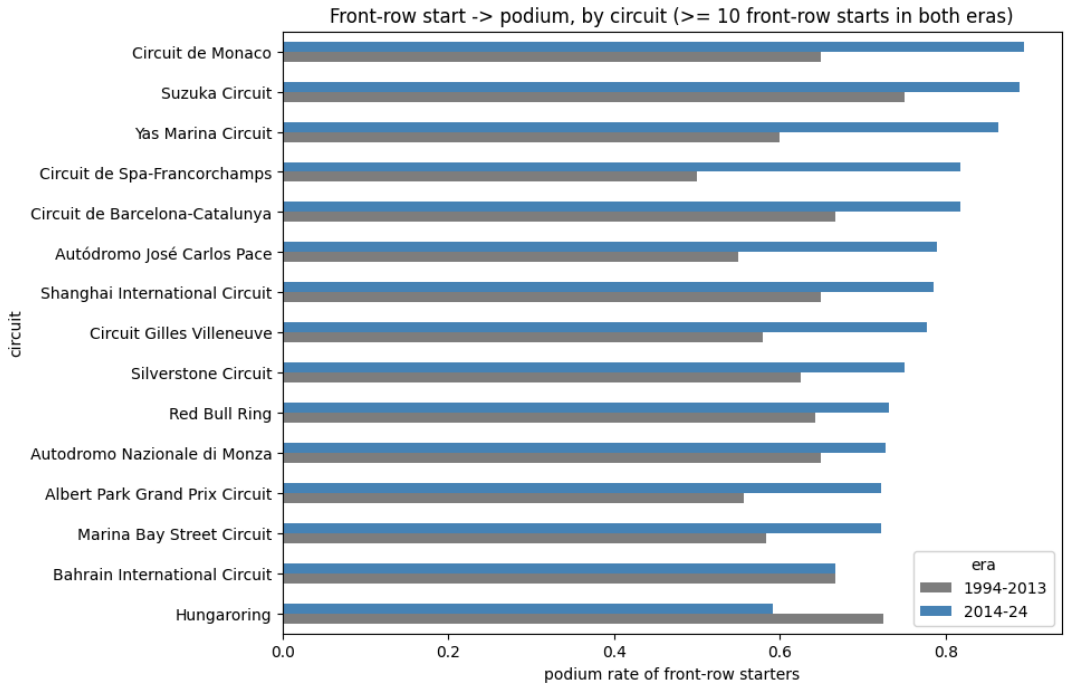
Qualifying gap to pole since 2003: median 1.72 %; podium finishers have a median gap of 0.33 % vs 1.99 % for the rest.



Best qualifying lap by era: almost the same median (86.6 s vs 86.8 s), so we use the relative gap to pole, not raw lap times.

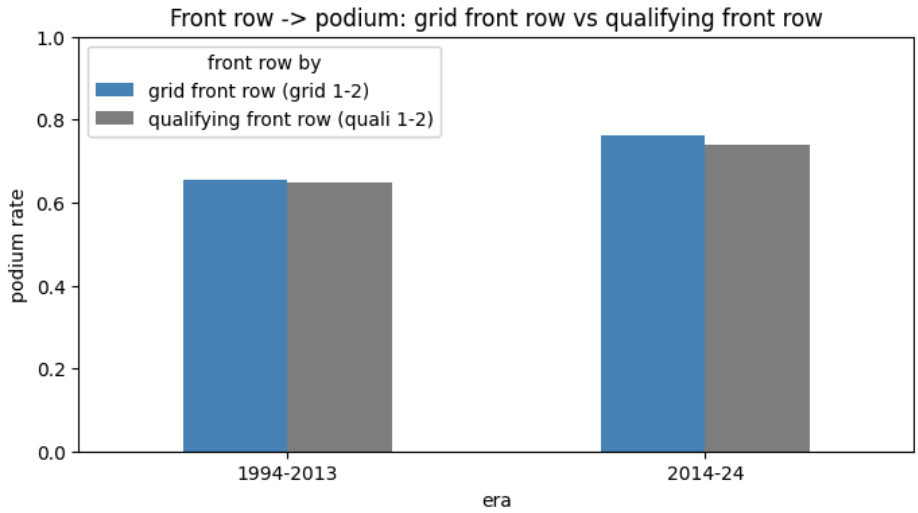
4. Data-engineering questions

Q1 — Which circuits convert a front-row start into a podium most often, and did this change after 2014?



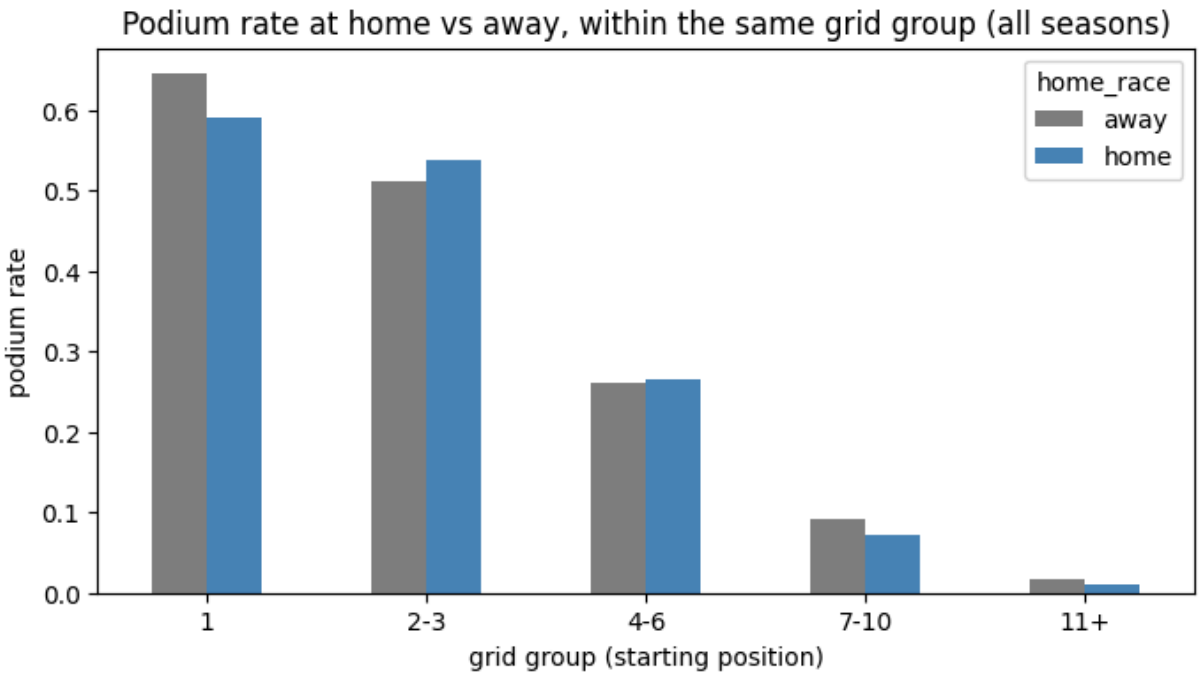
15 circuits with at least 10 front-row starts in both eras (1994–2013 vs 2014–24).

Answer. In 1994–2013 the best circuits were Suzuka (75 %), the Hungaroring (73 %), Bahrain and Barcelona (67 %). In 2014–24 they are Monaco (90 %), Suzuka (89 %) and Yas Marina (86 %). Conversion rose at 13 of the 15 circuits (all front-row starts: 64.6 % → 76.2 %), and the ranking changed almost completely (Spearman −0.04): the Hungaroring fell from 2nd to last, Spa rose from last to joint 4th. Using the qualifying front row instead of the grid gives almost the same rates (65 % / 74 %); the 37 front-row qualifiers who started further back (penalties) reached the podium only 30 % of the time. Grain: one row per start since 1994; 1,802 starts lack a qualifying row. *Cut-off:* grid and qualifying positions are both known before the race starts.



Grid front row vs qualifying front row: 65.5 % vs 64.9 % (1994–2013) and 76.2 % vs 74.0 % (2014–24). The grid slot, after penalties, is what counts.

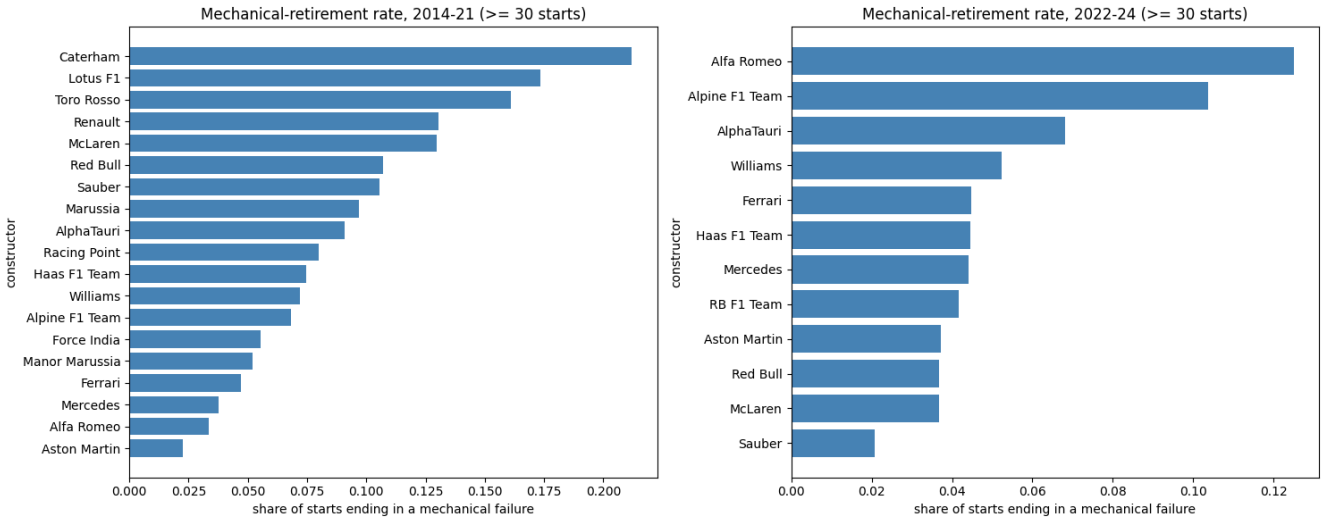
Q2 — Do drivers reach the podium more often at their home race, once starting position is controlled for?



Home vs away podium rate within the same grid group (all seasons, 1,932 home starts).

Answer: no. Without control, home drivers reach the podium *less* often (11.6 % vs 13.8 %). Within grid groups the differences are small and change sign (−5.4 / +2.7 / +0.5 / −2.1 / −0.6 points). A logistic regression *podium ~ home + grid group* gives a home odds ratio of 0.90 (bootstrap 95 % interval 0.76–1.07; 2003+ only: 0.82, 0.59–1.20). The interval includes 1, so there is no measurable home advantage. *Grain:* one row per starter, all seasons (24,584 rows). *Missing data:* none; every nationality and circuit country is mapped, and dual nationalities count as home in both countries. *Cut-off:* nationality, circuit and grid are all known before the race. This is an association, not a causal estimate.

Q3 — Which constructors have the highest mechanical-retirement rate, 2014–21 vs 2022–24?



Share of starts ending in a mechanical failure, constructors with at least 30 starts.

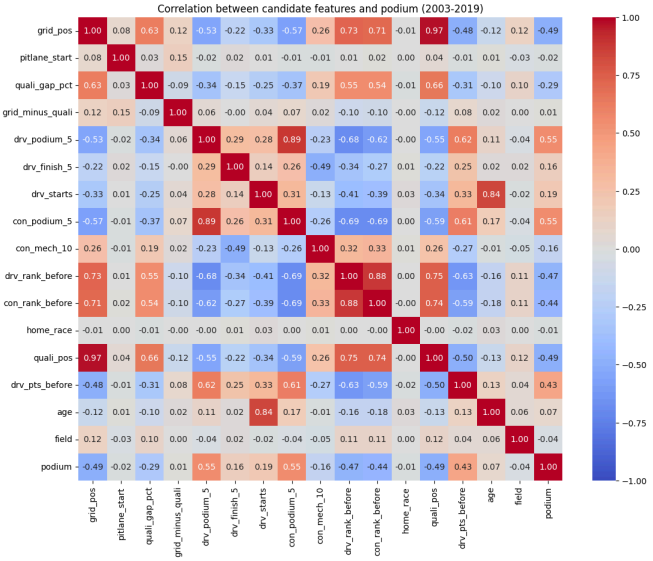
Answer. The overall mechanical-retirement rate fell from 8.9 % (2014–21) to 5.5 % (2022–24). Highest in 2014–21: Caterham 21.2 % (only 33 starts), Lotus F1 17.3 %, Toro Rosso 16.1 %, Renault 13.1 %, McLaren 13.0 %. Highest in 2022–24: Alfa Romeo 12.5 %, Alpine 10.4 %, AlphaTauri 6.8 %; lowest Sauber 2.1 %.

Counting "ambiguous" causes (e.g. wheel, tyre, wing) as mechanical does not change the top 5 (Spearman 0.98 / 0.95). Renamed teams (Toro Rosso → AlphaTauri → RB) are kept as separate constructorIds. *Grain:* one row per starter, rate per constructor and period (at least 30 starts). *Missing data:* every start has a status; the generic "Retired" (49 rows) names no cause and is not counted as mechanical, and crashes, driver errors and disqualifications are separate groups, as the brief's definition requires. *Cut-off:* `status` is a post-race field, used here only to answer the question and never as a feature of the race being predicted.

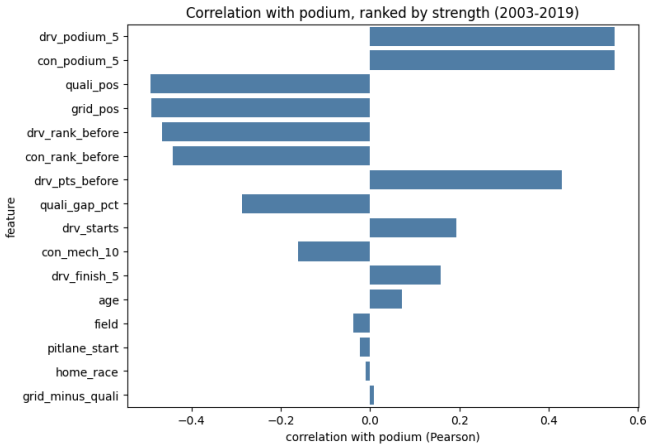
5. Features (all known before the race)

Group	Feature	Why it is valid before the race
Grid	`grid_pos`, `pitlane_start`	fixed after qualifying and penalties
Qualifying	`quali_gap_pct` (gap to pole, %), `grid_minus_quali` (penalty signal)	qualifying is finished before the race
Driver form	`drv_podium_5`, `drv_finish_5`, `drv_starts`	`shift(1)` before `rolling(5)`: only earlier races
Team form	`con_podium_5`, `con_mech_10`	same, per constructor and race
Standings	`drv_rank_before`, `con_rank_before`	taken from the previous round
Context	`home_race`	known from nationality and circuit

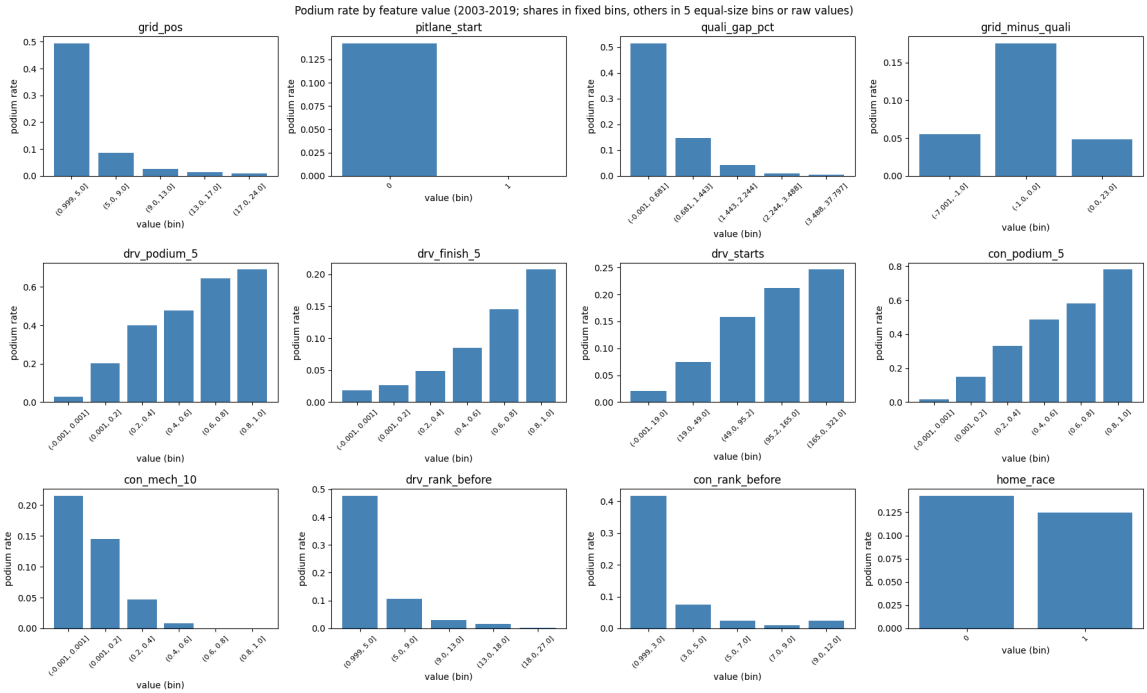
Dropped with evidence: `quali_pos` (0.97 correlated with grid), championship points (Spearman −0.86 with rank), age (0.84 with starts, 0.07 with podium), field size (6–24 starters, no relation with podium, r = −0.04). A leakage check on all 356 of Hamilton's starts shows that `drv_podium_5` equals the by-hand mean of his 5 earlier starts in every row, and that the rank comes from the previous round.



Correlation heatmap of all candidate features (2003–19). Dropped as near-duplicates: quali_pos (0.97 with grid) and age (0.84 with starts); points before the race repeat the rank (Spearman −0.86).



Correlation with podium, ranked: form and grid carry the signal; home race ≈ 0.



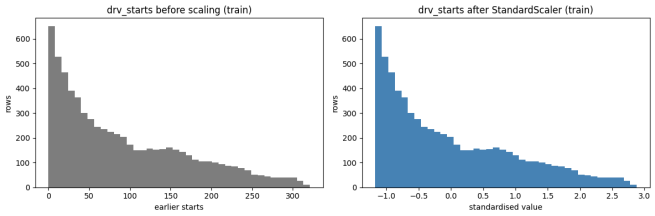
Effect of each feature: podium rate by feature value (2003–19). The effects are clear and mostly monotonic, e.g. no podium in the last 5 starts → very low podium rate.

6. Split and preprocessing

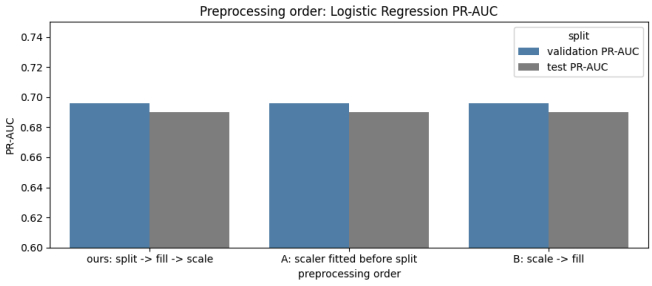
Split	Seasons	Rows	Races	Podium rate
Train	2003–2019	6,783	321	14.2 %
Validation	2020–2021	775	39	15.1 %
Test	2022–2024	1,352	68	15.1 %

We split by season, not randomly: a random split would put rows of the same race (and of later races) into training, so the model would see the future. All choices (settings, thresholds, the best model) use validation only; test is used once at the end. Preprocessing is fitted on train only: `quali_gap_pct` clipped at the train 99th percentile (12.46 %; 67 train, 5 validation and 25 test values clipped, maxima before 37.8 / 31.4 / 45.6 %; the median is unchanged), missing values (at most 7.1 %, mostly the rank at round 1) filled with the train median, then `StandardScaler` (scaling changes the range of each feature to mean 0 and standard deviation 1, but not the shape of its distribution).

Preprocessing-order experiments (Logistic Regression, validation / test PR-AUC): ours (split → fill → scale) 0.6961 / 0.6899; (A) scaler fitted before the split 0.6960 / 0.6899; (B) scale → fill 0.6961 / 0.6898. The scores hardly move, but A lets test-season information into the scaler (mean of `drv_starts` 92.9 → 103.4) and hides the drift, and B leaves the filled feature off-centre (std 0.964 instead of 1). We therefore keep split → fill → scale, fitted on train.



Effect of scaling on `drv_starts` (train): the range changes, the shape does not.



Preprocessing-order experiments: Logistic Regression PR-AUC for the three orders (almost identical).

7. Models and results

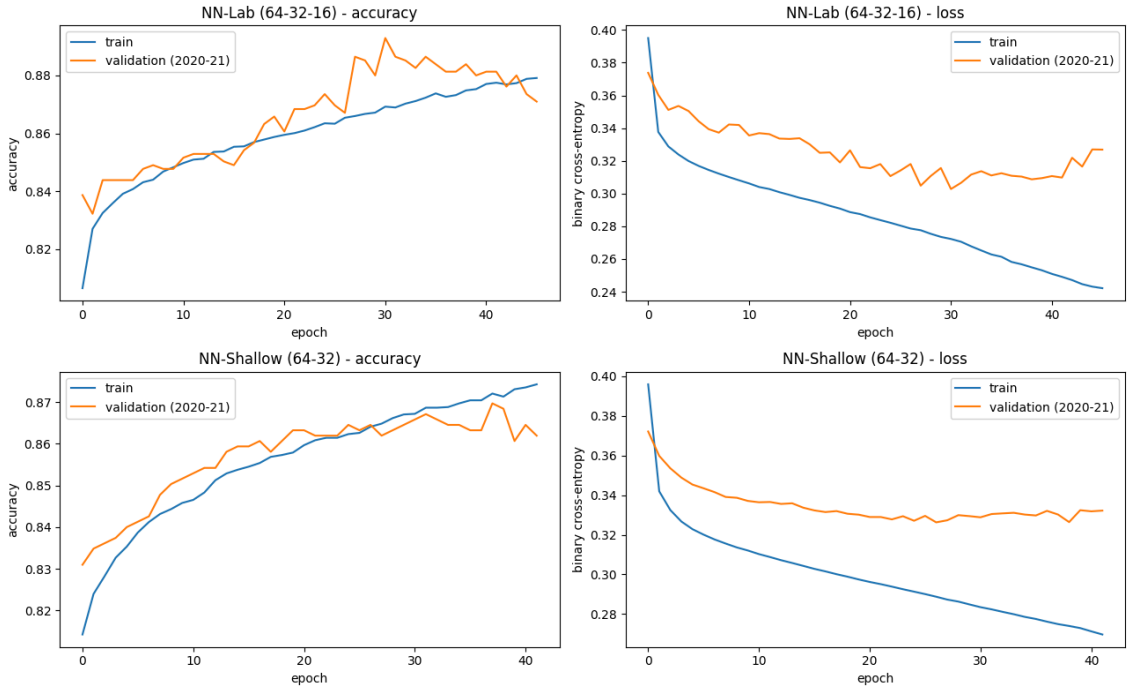
All models use the same rows and the same 12 features, so their scores are directly comparable.

Model	How it works	Limitations
Logistic Regression (baseline)	One weight per scaled feature, summed and passed through a sigmoid; `class_weight='balanced'` for the 1-in-7 imbalance.	Only a straight boundary (no interactions); correlated features share weights unstably.
Random Forest (default → tuned)	100 decision trees on random rows/features, votes averaged; tuned on validation over `max_depth` × `min_samples_leaf` → depth 10, leaf 20.	Deep trees memorise the training seasons (default: train PR-AUC 1.000); harder to explain than weights.
Keras NN (Lab 3)	NN-Shallow 64→32 (the required 1–2 hidden layers) and NN-Lab 64→32→16 (reference); ReLU, sigmoid output, Adam, binary cross-entropy, class weights 0.58 / 3.52, EarlyStopping (patience 15) on 2020–21.	More settings to choose, slight seed dependence, hardest to explain; fitted 2020–21 most closely.

Metrics. Primary: PR-AUC. Podiums are rare (~15 %), and PR-AUC measures how well the true podium finishers are ranked at the top, without being flattered by the many easy "no podium" rows, as ROC-AUC can be. **Secondary: F1** at the threshold chosen on validation (the best-F1 value), because a yes/no decision needs a balance of precision and recall. Thresholds and hyperparameters use validation only; test is used once.

Model	Split	Thr.	ROC-AUC	PR-AUC	F1	Precision	Recall	Accuracy
Logistic Regression	validation	0.72	0.924	0.696	0.699	0.659	0.744	0.903
	test		0.932	0.690	0.657	0.578	0.760	0.880
RF default	validation	0.24	0.919	0.665	0.695	0.634	0.769	0.898
	test		0.926	0.661	0.654	0.546	0.814	0.870
RF tuned (chosen)	validation	0.80	0.933	**0.719**	0.706	0.750	0.667	0.916
	test		0.934	0.684	0.642	0.681	0.608	0.898
NN-Lab (64-32-16)	validation	0.65	0.923	0.708	0.717	0.664	0.778	0.907
	test		0.908	0.639	0.647	0.566	0.755	0.876
NN-Shallow (64-32)	validation	0.66	0.924	0.718	0.706	0.652	0.769	0.903
	test		0.920	0.652	0.645	0.552	0.775	0.871
Rule: grid ≤ 3	validation	—	0.820	0.530	0.695	0.698	0.692	0.908
	test		0.793	0.474	0.649	0.650	0.647	0.894

Training performance (PR-AUC / F1 on 2003–19): LR 0.670 / 0.664 · RF default 1.000 / 0.945 · RF tuned 0.755 / 0.679 · NN-Lab 0.742 / 0.705 · NN-Shallow 0.735 / 0.691. The default forest's perfect training score next to its weaker validation score is clear overfitting, which tuning reduces (train–validation gap 0.335 → 0.036).



Neural-network training curves: training loss keeps falling while validation loss flattens and rises; early stopping keeps the best epoch (31 for NN-Lab, 27 for NN-Shallow).

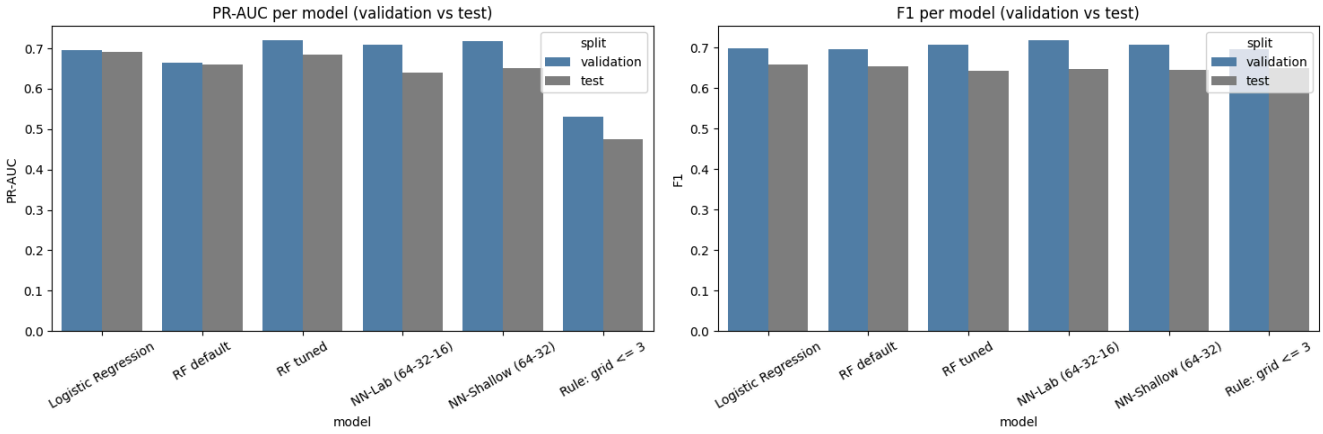
Neural-network details

Both networks take the 12 scaled features as input and are trained with Adam, binary cross-entropy, batch size 32, at most 200 epochs, EarlyStopping (val_loss, patience 15, best weights restored) and class weights 0.58 / 3.52. Layers from `model.summary()`:

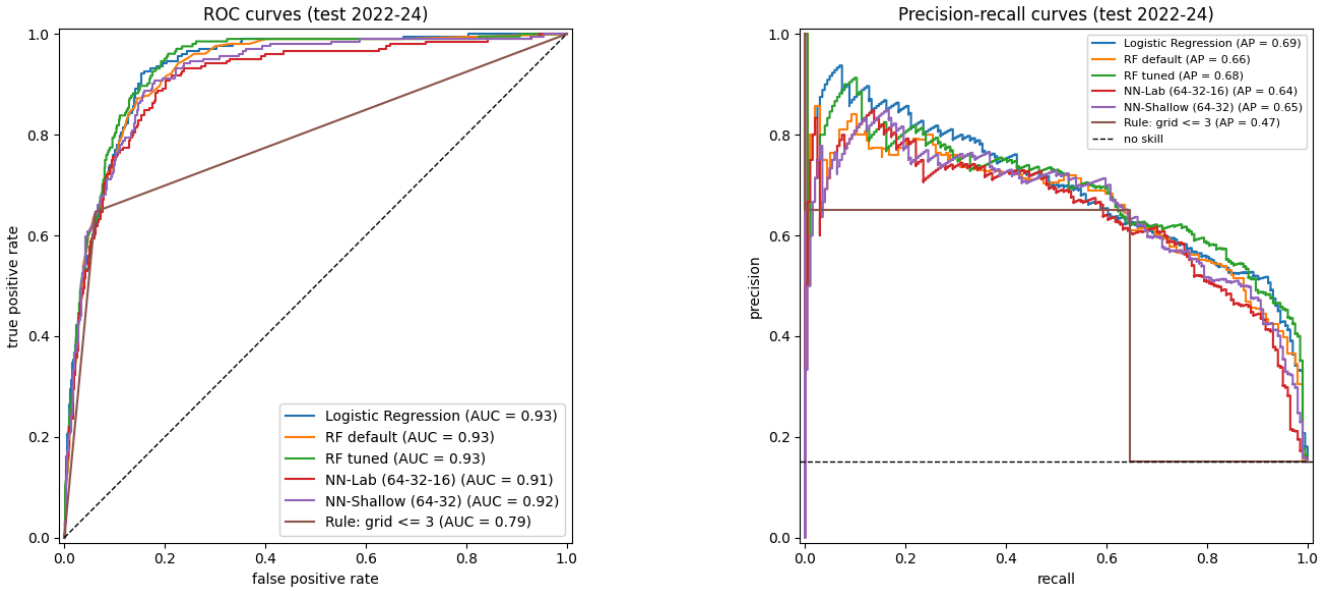
Network	Layer	Output	Weights
NN-Lab (64-32-16)	Dense, ReLU	64	832
	Dense, ReLU	32	2,080
	Dense, ReLU	16	528
	Dense, sigmoid	1	17
	Total		**3,457**
NN-Shallow (64-32)	Dense, ReLU	64	832
	Dense, ReLU	32	2,080
	Dense, sigmoid	1	33
	Total		**2,945**

Network	Epochs run	Best epoch (kept)	Train loss	Val loss	Train accuracy	Val accuracy
NN-Lab (64-32-16)	46	31	0.2723	0.3028	0.8692	0.8929
NN-Shallow (64-32)	42	27	0.2888	0.3263	0.8641	0.8645

The notebook prints one line per epoch (loss, accuracy, val_loss, val_accuracy) and the full per-epoch table in section 14.1. Training stops 15 epochs after the best validation loss and the best weights are restored.

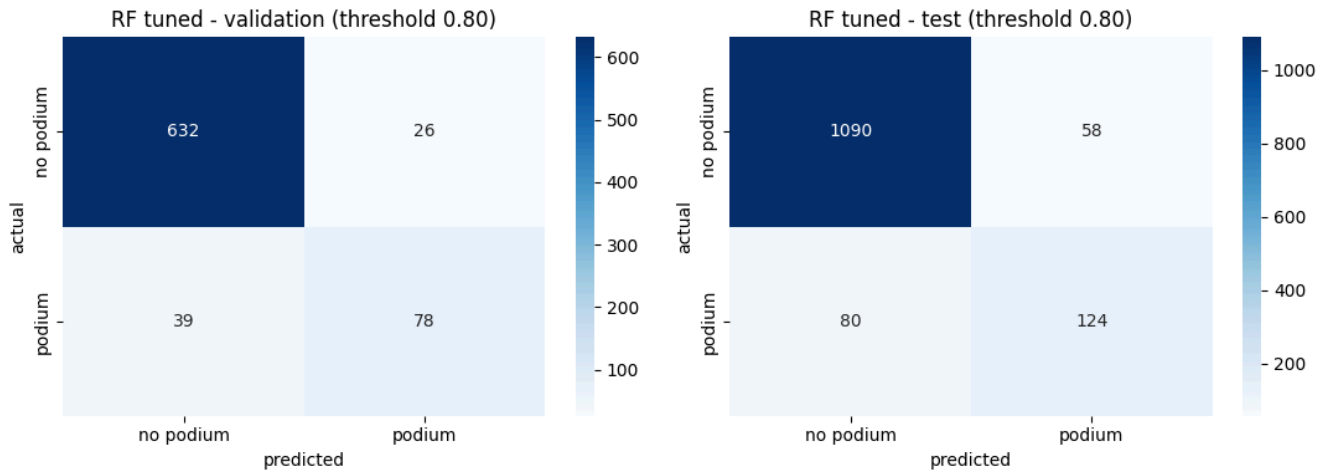


PR-AUC and F1 per model, validation vs test.



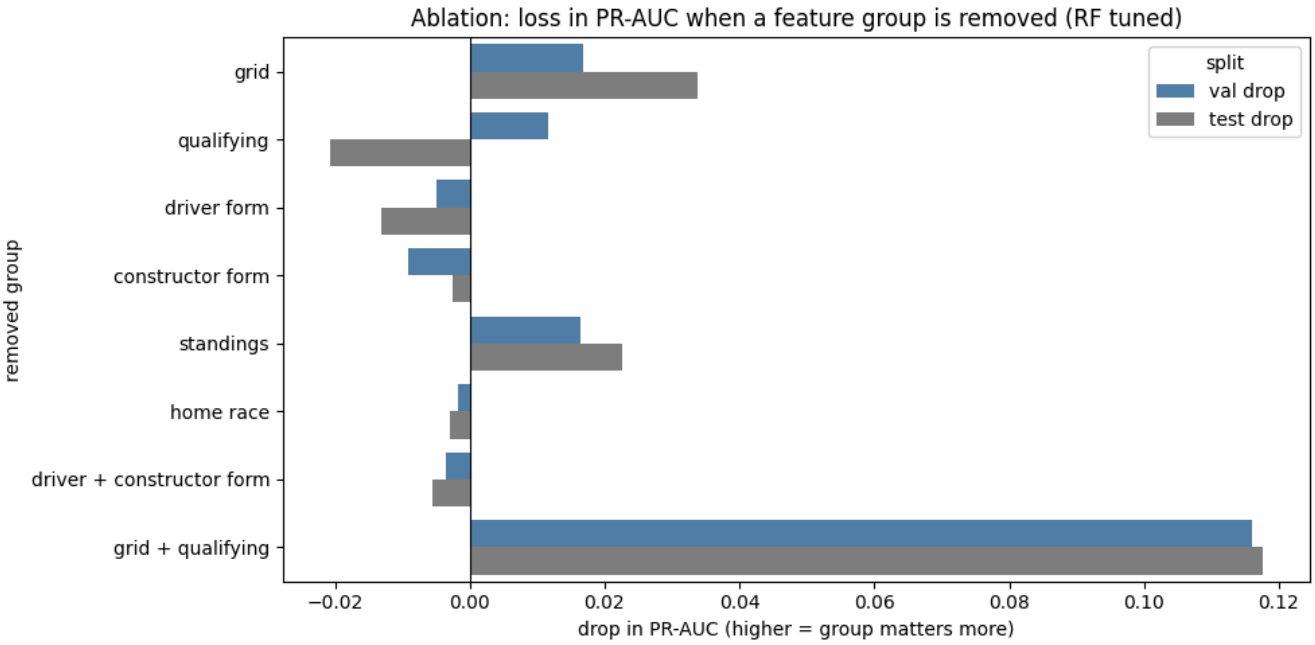
ROC and precision-recall curves on test 2022–24.

Reading the results. RF tuned is chosen because it is best on validation; on test the baseline logistic regression is slightly better (0.690 vs 0.684), and the networks drop the most (they fitted 2020–21 more closely). All models clearly beat the grid rule in *ranking*, while at a single threshold their F1 is close to the rule's: the gain is in how confidently they order the drivers. With its strict threshold of 0.80, RF tuned finds 124 of the 204 test podiums with 58 false alarms.



Confusion matrices of RF tuned (threshold 0.80) on validation and test.

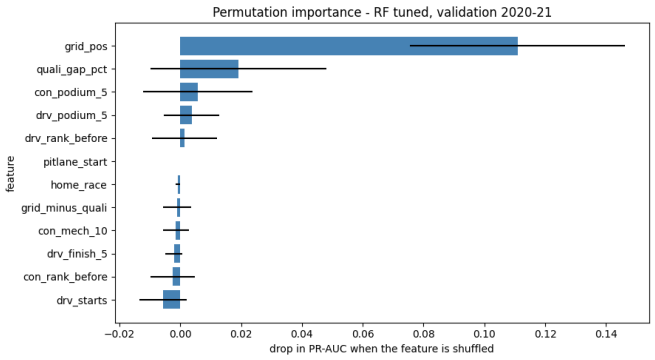
8. Ablation



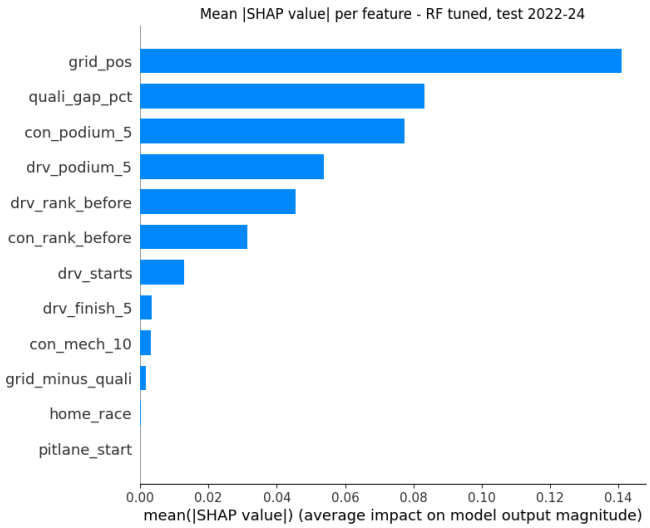
Drop in PR-AUC when a feature group is removed and RF tuned is retrained.

Grid (−0.017 val / −0.034 test) and standings (−0.017 / −0.023) are the only single groups whose removal hurts on both splits. Removing grid *and* qualifying together costs 0.116 / 0.118, far more than either alone, because both describe Saturday pace and each covers for the other. Driver form, team form and home race can each be removed with almost no loss: their information is already in the standings and the grid.

9. Explainable AI

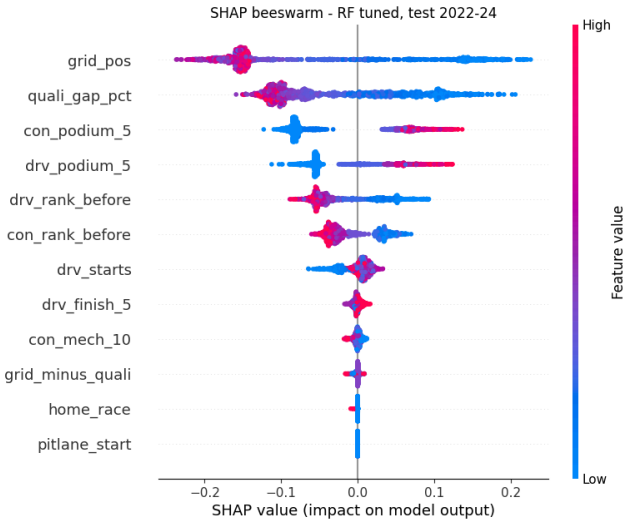


Permutation importance on validation (drop in PR-AUC when a feature is shuffled).

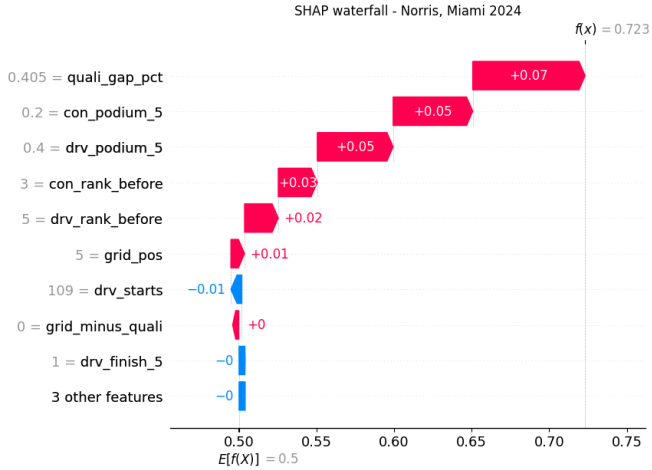


Mean |SHAP value| per feature on test.

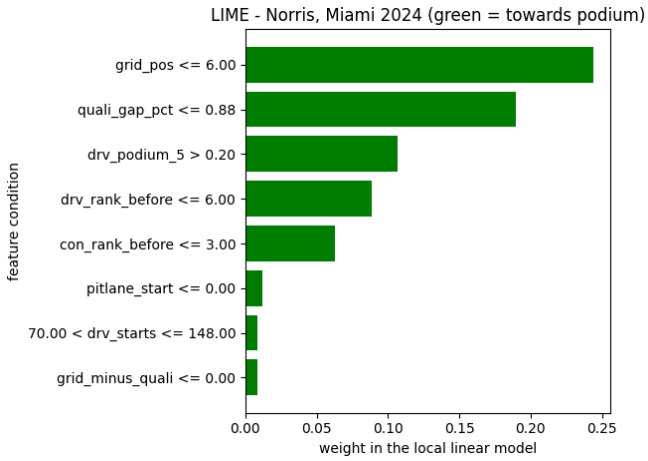
Global. Permutation importance puts `grid_pos` far ahead (0.111), then `quali_gap_pct` (0.019); correlated form and rank features look unimportant because their partner takes over when one is shuffled. SHAP spreads the credit more fairly: grid (0.14), qualifying gap (0.08), team podium share (0.08), driver podium share (0.05), then the ranks. The beeswarm shows sensible directions: a low grid slot, a small gap to pole, recent podiums and a good championship rank all push towards a podium.



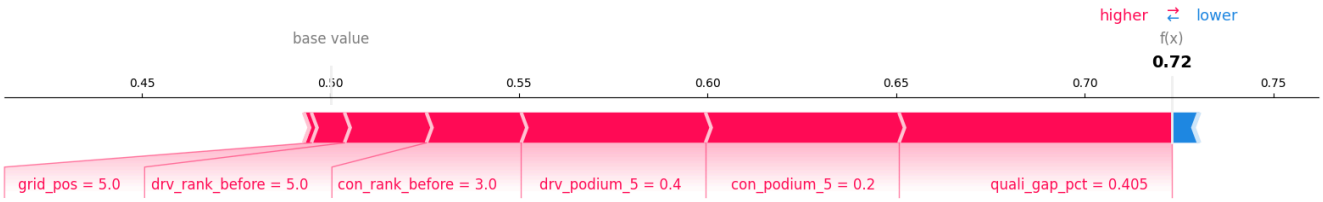
SHAP beeswarm (test 2022–24): red = high feature value.



SHAP waterfall: Norris, Miami 2024 (started P5, won).



LIME for the same driver and race.



SHAP force plot for the same prediction: red features push the probability up from the base value 0.50 to 0.72.

Local. For Lando Norris at the 2024 Miami GP the model gives 0.72 (base 0.50), just below the 0.80 threshold, so the label is "no podium" although he won. SHAP credits the small qualifying gap (+0.07) and McLaren's and his own recent podiums (+0.05 each); P5 itself adds only +0.01. LIME agrees on the direction (grid ≤ 6 +0.24, quali gap +0.19, recent podiums +0.11) but gives the grid more weight because it explains with simple rules around this one case.

> **Importance is not causation.** These methods show what the model uses, not what causes a podium. A high SHAP value for team podium share means the model relies on recent team results; it does not mean recent podiums make a car fast. Both are driven by things we do not measure (car performance, budget, driver skill).

10. Inference

`predict_podium(record)` takes a dictionary with any of the 12 features, applies the training preprocessing (clip, fill with train medians) and returns the probability, the yes/no label and the list of filled features. Norris, Miami 2024: complete record 0.723; with qualifying missing 0.582 (more cautious); a hand-made P12 midfield driver with no recent podiums: 0.038.

11. Conclusions and limitations

- Grid position and qualifying pace carry most of the pre-race signal, followed by the standings and recent form; there is no home advantage.
- A simple logistic regression is almost as good as the forest and the networks; the extra model complexity buys little on this data.
- **Qualifying coverage:** complete only from 2003, so earlier seasons cannot be used for training.

- **Era and regulation changes:** the grid became more decisive and reliability improved, so patterns from 2003–19 drift in 2020–24 (visible in the drop from validation to test).
- **Threshold:** the 0.80 threshold from 2020–21 is strict for 2022–24; the probability is more useful than the yes/no label.
- **Shared drives and renamed teams:** shared drives (1950–1964) were reduced to one row per driver and race; renamed teams are separate constructors, which breaks their form history at each rename.
- **Missing context:** no weather, safety cars, tyre strategy, upgrades or late penalties, which explain many surprise podiums.

12. Bonus: a better cleaning and split (V13)

Sections 2–7 make one choice for each cleaning and split step. For the bonus we tested whether other choices give a better model. We ran **13 copies of the notebook (V01–V13)**; each changes one thing and trains the same three models (Logistic Regression, the tuned Random Forest, NN-Shallow 64→32), with the threshold chosen on validation and the model chosen by **validation** PR-AUC. All runs: kaggle.com/code/amreladawy2/f1-ms1-bonus-cleaning-and-split. The best setup (V13) is also run at the end of the main notebook (section 21).

Copy	What changes	Chosen	Val PR-AUC	Test PR-AUC	Test F1
V01	nothing (the submitted pipeline above)	RF	0.719	0.684	0.642
V02	missing values filled with their F1 meaning	RF	0.719	0.688	0.678
V03	training rows with a missing value dropped	RF	0.716	0.694	0.652
V04	no clipping of the qualifying gap	RF	0.722	0.685	0.624
V05	3 weakest features dropped ($\|\rho\| \le 0.02$)	RF	0.718	0.687	0.670
V06	2 redundant constructor features dropped	RF	0.724	0.687	0.662
V07	**leak demo:** standings *after* the race	RF	0.760	0.736	0.671
V08	random 70/15/15 split of rows (as in the labs)	RF	0.713	0.702*	0.678*
V09	random 70/15/15 split of whole races	RF	0.784	0.683*	0.680*
V10	train on 2010–2019 only	RF	0.727	0.689	0.619
V11	validation 2019–21 (train 2003–18)	RF	0.741**	0.698	0.662
V12	V06 + V11	RF	0.748**	0.695	0.679
V13	**V12 + V02 (recommended)**	**RF**	**0.750****	**0.692**	**0.686**

* own random test set, not comparable. ** validation 2019–21, comparable only with V11–V13. Test = 2022–24 for all other rows.

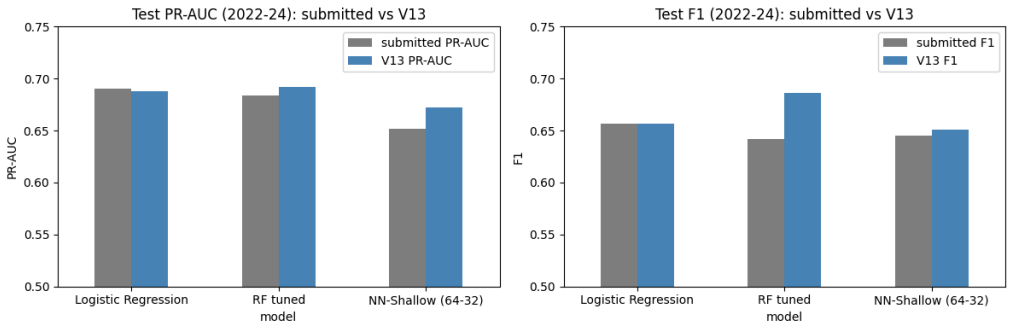
What the grid shows. Leakage is by far the biggest effect: standings taken after the race (V07) lift test PR-AUC to 0.736, which no honest setup reaches, and confirms cleaning rule 8. Random splits give an over-optimistic validation score (V09: 0.784 on validation, 0.683 on test) because eras are mixed, and a random row split (V08) puts 822 races into more than one split. The honest cleaning choices move PR-AUC by about 0.01: filling gaps by meaning helps all three models, dropping the redundant constructor features gives the best validation score of the cleaning changes (0.724) with 2 fewer inputs, while no clipping and dropping the weak features change almost nothing. Training only on 2010–19 (V10) loses 37 % of the rows without a clear gain (test F1 falls to 0.619).

The V13 setup

Change	What it does	Why
Fill gaps by their F1 meaning	rank before round 1 → field size of that race; debut → 0 podium history; no qualifying time → the clip value (train 99th percentile); no qualifying row → 0 penalty; anything else → train median	The median says "an average driver" (train median rank 11), but a missing rank means "no standing yet" (463 train rows, filled with 21 on average) and a debut means "no podium history".
Drop `con_podium_5`, `con_rank_before`	10 features instead of 12	They copy `drv_podium_5` and `drv_rank_before` (ρ 0.89 / 0.88).
Validation 2019–2021	train 2003–18 (6,363 rows, 300 races), validation 2019–21 (1,195 rows, 60 races), test 2022–24 unchanged (1,352 rows, 68 races)	60 validation races instead of 39 make the threshold and the forest settings less noisy; still a season split with no shared races.

Model (V13)	Split	Thr.	ROC-AUC	PR-AUC	F1	Precision	Recall	Accuracy
Logistic Regression	validation	0.72	0.928	0.712	0.721	0.653	0.806	0.906
	test		0.931	0.688	0.657	0.587	0.745	0.882
RF tuned (chosen)	validation	0.68	0.939	**0.750**	0.726	0.668	0.794	0.910
	test		0.936	0.692	0.686	0.601	0.799	0.890
NN-Shallow (64-32)	validation	0.54	0.928	0.700	0.715	0.632	0.822	0.901
	test		0.917	0.672	0.651	0.567	0.765	0.876

RF tuned is again chosen on validation (max_depth 15, min_samples_leaf 20); NN-Shallow stopped after 68 epochs (best epoch 53, val_loss 0.2813). These numbers are identical in the bonus copy and in section 21 of the main notebook.



Test 2022–24, submitted pipeline (grey) vs V13 (blue): PR-AUC and F1 per model.

Why V13 won. (1) It was chosen without the test set: V11–V13 share the 2019–21 validation seasons and V13 scores highest there (RF 0.750 vs 0.748 and 0.741). (2) The split stays honest: by season, no shared races, a larger validation set. (3) The cleaning follows the meaning of the data. (4) It is simpler: 10 features, no loss. (5) On the unseen 2022–24 seasons RF tuned improves from PR-AUC 0.684 to **0.692** and F1 0.642 to **0.686**, finding 163 of the 204 test podiums instead of 124 (recall 0.608 → 0.799). The trade-off is precision (0.681 → 0.601), because the threshold chosen on validation is lower (0.68 instead of 0.80). NN-Shallow gains the most PR-AUC (0.652 → 0.672); the logistic regression barely moves (0.690 → 0.688).

> **Caveat.** The gains are small (about 0.01 PR-AUC). V11 alone has a slightly higher RF test PR-AUC (0.698), but it would not have been picked on validation. The main lesson is that the submitted pipeline is already close to what these features allow; the two choices that really change the numbers are leakage and the kind of split.